

DevOps Project-2 — Amazon Prime Clone

Full Step-by-Step Execution Guide

This guide takes you from an empty AWS account to a running, monitored Amazon Prime clone on EKS — with a fully automated Jenkins pipeline and Argo CD GitOps. Every step has copy-paste commands.

Cost warning: This creates real AWS resources (EKS control plane ~\$0.10/hr, 2× t3.medium nodes, a NAT gateway, ELBs). Budget a few dollars for a day of testing and run `terraform destroy` at the end.

0. Prerequisites

- An AWS account + an IAM user with **AdministratorAccess** (for the demo).
- AWS access key + secret (`aws configure`).
- A GitHub account and this repo pushed to your own GitHub.
- A free **TMDB API key** (the clone lists movies): <https://www.themoviedb.org/settings/api>
- A key pair to SSH into EC2.

Push this scaffold to your own GitHub first:

```
cd devops-project2
git init && git add . && git commit -m "init devops project-2"
git branch -M main
git remote add origin https://github.com/<your-org>/devops-project2.git
git push -u origin main
```

1. Launch the Jenkins / build server (EC2)

1. EC2 → Launch instance → **Ubuntu 22.04**, type **t2.large** (2 vCPU / 8 GB), 30 GB disk.
2. Security group inbound: `22` (SSH), `8080` (Jenkins), `9000` (SonarQube), `9090`.
3. SSH in and install everything:

```
ssh -i key.pem ubuntu@<EC2_PUBLIC_IP>

# clone your repo onto the box
git clone https://github.com/<your-org>/devops-project2.git
cd devops-project2

# install docker, aws cli, kubectl, terraform, helm, trivy, node, sonar-scanner, argocd
bash scripts/install-tools.sh
# log out & back in (docker group), then:
bash scripts/install-jenkins.sh      # prints the initial admin password
bash scripts/run-sonarqube.sh        # SonarQube on :9000
```

4. Configure AWS creds on the box:

```
aws configure      # enter your access key / secret / us-east-1 / json
```

2. Provision AWS infra with Terraform (EKS + ECR)

```
cd ~/devops-project2/terraform
cp terraform.tfvars.example terraform.tfvars # edit region/sizes if needed

terraform init
terraform plan
terraform apply -auto-approve # ~15-20 min (EKS takes a while)
```

When it finishes, wire up `kubectl` using the printed output:

```
aws eks update-kubeconfig --region us-east-1 --name prime-clone-eks
kubectl get nodes # should list 2 Ready nodes
```

Note the ECR URL from the output (e.g. `123456789012.dkr.ecr.us-east-1.amazonaws.com/prime-clone`). Put that account id into `Jenkinsfile` (`ACCOUNT_ID`) and into `kubernetes/deployment.yaml` (the `image:` line).

3. Install Argo CD (GitOps) on the cluster

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install

# wait for it to come up
kubectl -n argocd rollout status deploy/argocd-server

# expose the UI (quick way) and get the admin password
kubectl -n argocd patch svc argocd-server -p '{"spec":{"type":"LoadBalancer"}}'
kubectl -n argocd get svc argocd-server # note EXTERNAL-IP
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath="{.data.password}" | base64 -d; echo
```

Register the app (edit `argocd/application.yaml` repoURL to your repo first):

```
kubectl apply -f ~/devops-project2/argocd/application.yaml
```

Argo CD now watches `kubernetes/` in your repo and keeps EKS in sync.

4. Install monitoring (Prometheus + Grafana via Helm)

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

kubectl create namespace monitoring
helm install monitoring prometheus-community/kube-prometheus-stack \
  -n monitoring \
  -f ~/devops-project2/monitoring/monitoring-values.yaml

# wait for grafana, then grab the PUBLIC ELB URL
kubectl -n monitoring rollout status deploy/monitoring-grafana
kubectl -n monitoring get svc monitoring-grafana \
  -o jsonpath='{.status.loadBalancer.ingress[0].hostname}'; echo
```

The command prints a public URL like `a1b2c3...us-east-1.elb.amazonaws.com` — **this is the exact same kind of URL you saw in the video**. Open it in the browser.

Get the exact "Node Exporter Full" output

The `monitoring-values.yaml` in this repo **auto-imports** the Node Exporter Full dashboard (Grafana ID **1860**), so you don't have to import anything by hand.

1. Open the Grafana ELB URL → log in `admin` / password from the values file.
2. Left menu → **Dashboards** → open **Node Exporter Full**.
3. Top-right: set the time range to **Last 5 minutes** and auto-refresh to **5s**.
4. Use the **Host / instance** dropdown to pick your node (e.g. `10.0.x.x:9100`).

You'll now see the identical panels from the video — Pressure, CPU Busy, Sys Load, RAM Used, SWAP, Root FS, Uptime, plus the CPU Basic / Memory Basic time-series. node-exporter (running as a DaemonSet on every node) is what feeds these metrics.

If you'd rather keep Grafana private, set `grafana.service.type: ClusterIP` in the values file and reach it with:
`kubectl -n monitoring port-forward svc/monitoring-grafana 3000:80` then open `http://localhost:3000`.

5. Get the app source into `app/`

```
cd ~/devops-project2/app
git clone https://github.com/N4si/DevSecOps-Project.git . # or any React prime-clone
# now package.json sits next to the Dockerfile
cd ..
git add app && git commit -m "add app source" && git push
```

6. Configure Jenkins

Open `http://<EC2_PUBLIC_IP>:8080`, unlock with the initial password, install suggested plugins, then add these plugins (Manage Jenkins → Plugins): **Docker Pipeline**, **SonarQube Scanner**, **Pipeline: AWS Steps**, **NodeJS**, **Credentials Binding**.

Manage Jenkins → **Credentials** — add:

ID	Kind	Value
aws-creds	AWS credentials	your AWS key/secret
sonar-token	Secret text	SonarQube token (create in Sonar → My Account → Security)
tmdb-api-key	Secret text	your TMDb v3 key
github-creds	Username + password	GitHub username + PAT

Manage Jenkins → **System** → **SonarQube servers**: add one named exactly `SonarQube`, URL

`http://<EC2_PUBLIC_IP>:9000`, auth = the `sonar-token` credential.

In SonarQube, also add a **webhook** (Administration → Configuration → Webhooks):

`http://<EC2_PUBLIC_IP>:8080/sonarqube-webhook/` so the Quality Gate reports back.

7. Create the pipeline job & run it

1. Jenkins → **New Item** → *prime-clone* → **Pipeline** → OK.
2. Pipeline → *Definition* = **Pipeline script from SCM** → Git → your repo URL → branch `main` → *Script Path* = `Jenkinsfile`.
3. **Build Now**.

Watch the stages run: `Checkout` → `SonarQube` → `Quality Gate` → `npm build` → `Trivy FS` → `Docker build` → `Trivy image (gate)` → `Push to ECR` → `Update manifest`.

On success, Jenkins pushes a new `image:` tag to `kubernetes/deployment.yaml`, and **Argo CD auto-syncs** it to EKS.

8. See the app live

```
kubectl -n prime-clone get svc prime-clone      # note the LoadBalancer EXTERNAL-IP
```

Open `http://<EXTERNAL-IP>` — the Amazon Prime clone should load. Check Argo CD UI (all green) and Grafana for the cluster metrics.

9. Tear down (avoid AWS charges)

```
# remove app + monitoring + argocd first so ELBs are deleted
kubectl delete -f ~/devops-project2/kubernetes/ || true
helm uninstall monitoring -n monitoring || true
kubectl delete namespace argocd || true

# then destroy the infra
cd ~/devops-project2/terraform
terraform destroy -auto-approve
```

Troubleshooting

- **EKS nodes NotReady / 0 nodes**: check the node group in the EKS console; make sure subnets have the `kubernetes.io/cluster/...` tags (they do in `vpc.tf`).

- **docker: permission denied** on Jenkins: `sudo usermod -aG docker jenkins && sudo systemctl restart jenkins` .
- **Trivy gate fails the build**: that's the security gate working — fix/upgrade the flagged package, or (temporarily) relax severity in the `Trivy image` stage.
- **Argo CD OutOfSync**: confirm Jenkins pushed the manifest bump and that `argocd/application.yaml` repoURL/path point to your repo.
- **LoadBalancer stuck <pending>** : the AWS Load Balancer Controller / subnet ELB tags are needed; the public subnet tags in `vpc.tf` handle the classic ELB case.
- **ECR push denied**: the EC2/Jenkins identity needs `AmazonEC2ContainerRegistryPowerUser` .

Pipeline stage → tool → purpose

Stage	Tool	Purpose
Checkout	Git/GitHub	pull source
Code quality	SonarQube	SAST, bugs, code smells, quality gate
Build	npm	install deps, build the React app
Dependency scan	Trivy (fs)	known CVEs in dependencies
Image build	Docker	package app on nginx
Image scan (gate)	Trivy (image)	block CRITICAL/HIGH vulns
Registry	AWS ECR	store the immutable image
Deploy	Argo CD	GitOps sync to EKS
Runtime	AWS EKS + Pod	run the container
Monitoring	Prometheus + Grafana	metrics + dashboards
Infra	Terraform	provision EKS/ECR/VPC/IAM